bit.ly/4bSxADE

Lecture 5

Effective Abstractions (Principles, Part 1)

Example Code and Exercises

https://github.com/eecs498-software-design/lecture

Announcements

- Project 1 due tonight Mon Jan 26 at 11:59pm
  - Make sure all code is committed and pushed to `main`.
  - Detailed submission instructions in the project spec.
  - Remaining feature demos in lab this Friday Jan 30.
- Project 2
  - Released tomorrow Tue Jan 27.
  - We'll confirm groups via email and provision a new group GitHub repo.
- Lecture participation
  - Everyone gets credit for lectures 1-4. Attendance starting today in lec 5.

Video

Dimensions of Abstractions

These are all properties of the **interface** and **behavior** of the abstraction, not its implementation.

Abstractions can be qualified and assessed by:

- **Expressive Power**
- **Complexity**
- **Obscurity**
- **Depth** (Surface-to-Volume Ratio)

Much of software design is rooted in optimizing and making tradeoffs between these properties to create **effective abstractions** for a particular system.

*We might also consider **performance**. These trade-offs are often made at the language level.

Analogy – Burrito Restaurant

- **Expressive Power** What range of things you can do?

  Low – The only choices are 5 premade classic burritos.
  High – You can order and customize to your heart's content.

- **Complexity** How complicated is it to understand/use?

  Low – They kindly walk you through all your choices and provide suggestions.
  High – You must order in BNF* and must say "guac it up" if you want guacamole.

- **Obscurity** Is there hidden knowledge you might need?

  Low – They give you a warning before your toppings exceed burrito capacity.
  High – They silently charge you $1 extra at checkout for each salsa after the first two.

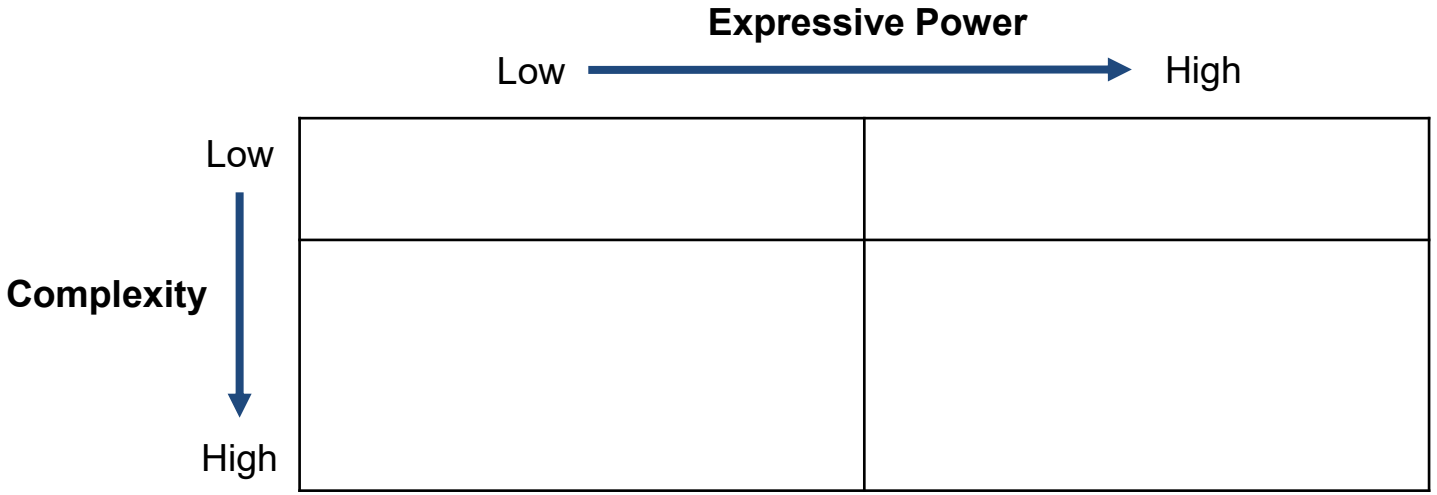
- **Depth** (Surface-to-Volume Ratio) How much complexity does it hide?

  Low – You order individual toppings and have to make the burrito yourself.
  High – You can save an order as a favorite for quick ordering next time.

**Burrito-Normal Form*

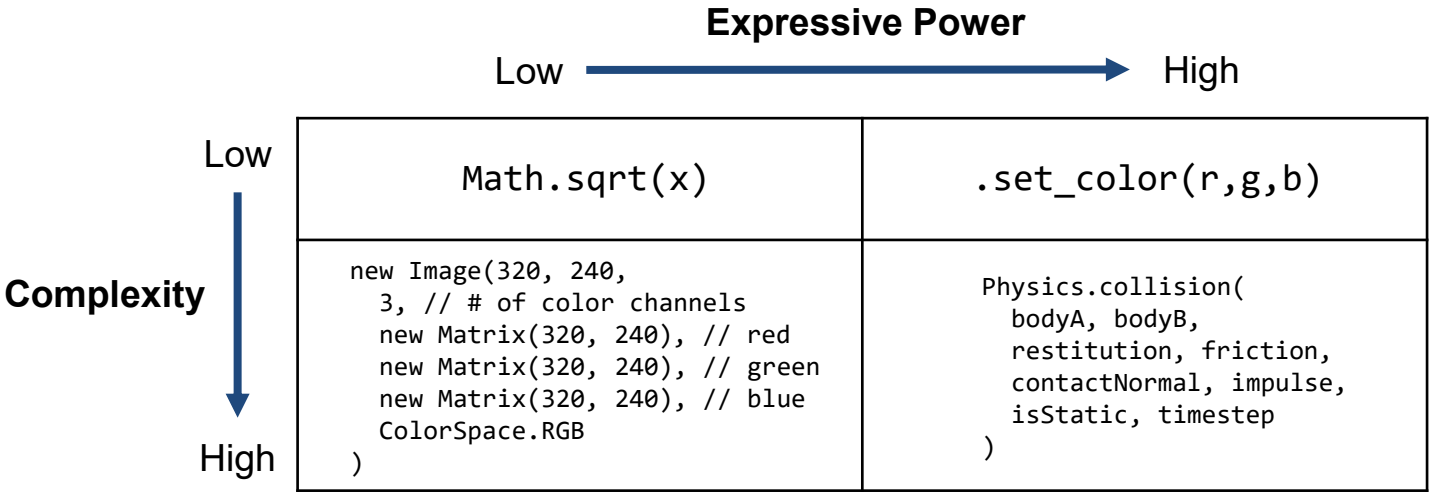
Complexity and Expressive Power

- Expressive Power — What range of things you can do?
- Complexity — How complicated is it to understand/use?



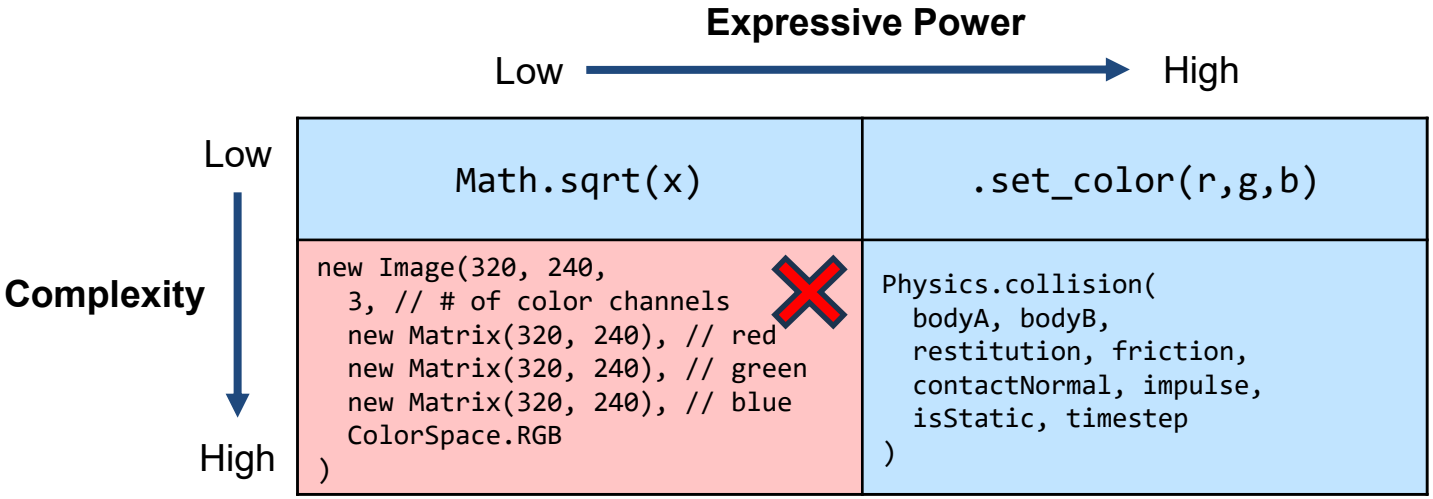
Complexity and Expressive Power

- Expressive Power — What range of things you can do?
- Complexity — How complicated is it to understand/use?



Complexity and Expressive Power

- Expressive Power — What range of things you can do?
- Complexity — How complicated is it to understand/use?



Video

Obscurity and Opacity

- Obscurity **Is there hidden knowledge you might need?**
  - Aim to *minimize* abstraction obscurity.
  - We may trade increased obscurity for an improvement elsewhere.

- Opacity **Is the knowledge you don't need appropriately hidden.**
  - We generally like opacity.
  - These are details you don't need – that's a benefit of abstraction!

Obscurity Tradeoffs

- Obscurity **Is there hidden knowledge you might need?**
  - We may trade increased obscurity for an improvement elsewhere.

Higher Complexity No Obscurity	Low Complexity Some Obscurity

`// REQUIRES: The vector has enough space in`
`//  its underlying buffer. If you need`
`//  more space, make sure to .reserve().`
`// COMPLEXITY: Worst case O(1).`
`void vector<T>::push_back();`
	`// REQUIRES: Nothing.`
	`// If there isn't enough space, the underlying`
	`// buffer is reallocated with 2x capacity.`
	`// COMPLEXITY: Worst case O(N), amortized O(1).`
	`void vector<T>::push_back();`

Obscurity Tradeoffs

- Obscurity **Is there hidden knowledge you might need?**


```
// REQUIRES: Nothing.
// If there isn't enough space, the underlying
// buffer is reallocated with 2x capacity.
// WARNING! May invalidate iterators on a grow.
// COMPLEXITY: Worst case O(N), amortized O(1).
void vector<T>::push_back(const T &val);
```

```
void add_max_to_back(const vector<T> &vec) {
  auto it = max_element(vec.begin(), vec.end());
  vec.push_back(*it);
}
```

Depth

- Depth **How much complexity does it hide?**
  - Aim to *maximize* abstraction depth.
  - A deep abstraction will make your life easier.

```
// Not so Complex, Expressive, Deep
circle.config({
  color: { r, g, b },
  radius: 4,
  // other configs optional
);
```

```
// Hides mathematical complexity. How
// do you even compute a square root?
const y = Math.sqrt(x);
```

```
// Fairly shallow – but that's ok
circle.set_color(r, g, b);
circle.set_radius(r, g, b);
circle.set_outline_width(r, g, b);
```

```
// Deeper – is it better?
circle.set_color_and_radius(r, g, b, rad);
```

```
// Even deeper – is it better?
circle.set_color_and_radius_and_outline_widt
h(r, g, b, rad, t);
```

Composition (Depth as Surface-to-Volume Ratio)

- ***Compose*** *several shallower abstractions along (mostly)* ***orthogonal*** *dimensions.*
- Example: Higher-order array functions that accept a predicate.

	`is_even()`	`is_prime()`	`(x) => x === 0`	`...`
`filter()`				
`count_if()`	`count_evens()`			
`replace_if()`				
`...`				

- Shallow(er) pieces, deep compositions.
- Not 100% free! Complexity may move to coupling between pieces… (next time)

Individually, `count_evens()` is a deeper abstraction than `count_if()` + `is_even()`

But… consider the library of these functions holistically!
A: *n* x m functions
B: *n* functions + *m* predicates

A and B have equal "volume" of functionality, but B has a smaller interface surface.

Custom predicates arguably add expressive power!

Low-Depth Abstractions

- Abstractions hide complexity
  - More depth = more hidden complexity
- Abstractions aren't free
  - Complexity and obscurity are never truly zero. At a minimum, there's some cognitive overhead.
- Consider removing **low-depth abstractions**, when the benefit doesn't justify the cost.

```
class Circle {
  private rad: number;
  getRadius(): number { return this.rad; }
  setRadius(rad: number): void { this.rad = rad; }
};
```

```
class Circle {
  private rad: number;
  getRadius(): number { return this.radius; }
};
```

```
class Circle {
  readonly public rad: number;
};
```

```
let email = "";
email += formatSubtotalLine(order.subtotal);
email += formatTaxLine(order.tax);
email += formatTotalLine(order.total);
```

```
function formatSubtotalLine(subtotal: number): string {
  return `Subtotal: ${formatCurrency(subtotal)}`;
}
```

Case Study: Luxon Date/Time Library

```
import { DateTime } from 'luxon';


// There is so much complexity hidden away here, e.g.
// how many days per month, leap years, daylight savings time, etc.
const result = DateTime.now()    // Local system time
  .setZone('America/New_York')  // Map to a specific timezone
  .plus({ months: 7 })          // 7 months from now
  .setLocale('fr')              // Print using a French locale
  .toLocaleString(DateTime.DATETIME_FULL);


console.log(result); // e.g., "26 août 2026 à 07:10 UTC−4"
```

Design Pattern: Facade

- The Façade pattern generally trades:
  - reduced expressive power
  - or increased obscurity

- To gain reduced complexity

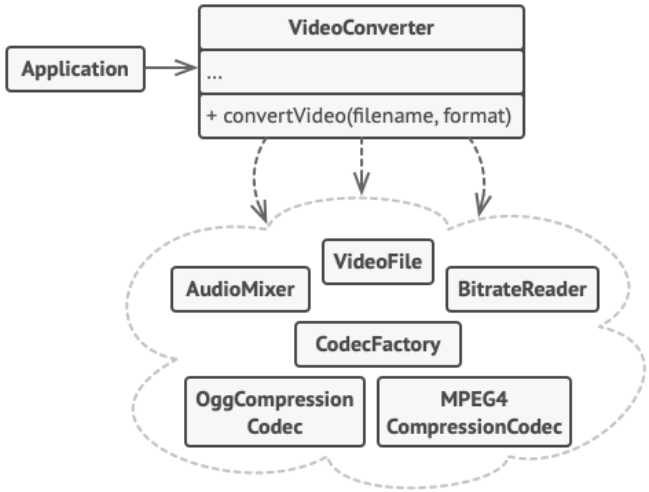


Image credit: https://refactoring.guru/design-patterns/facade

Video

KISS and YAGNI

But lad, this time's different.
Mrs. Puff needs this!

- **KISS** Keep It Simple, Stupid

  People often *under*estimate the cost of complexity.

- **YAGNI** You Ain't Gonna Need It

  People often *over*estimate the value of expressiveness or composability.